

AI Security Red Team Assessment Report

Project Topic: AI Red Teaming, Adversarial Attacks, and Model Defense

Target Model: SimpleCNN (PyTorch Convolutional Neural Network trained on MNIST)

Name: Kanrabit Chala

Date: 9/29/2026

To:

1. Introduction and Project Overview

Artificial Intelligence models are increasingly used in important software, but they have unique security weaknesses. In this project, we performed an **AI Security Assessment**.

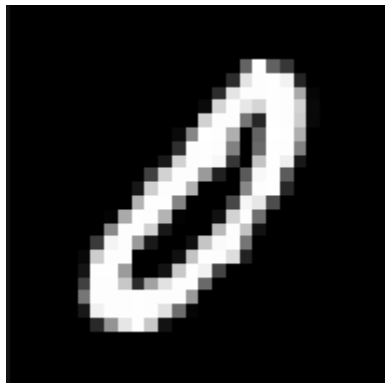
- **The Goal:** First, act as a "Red Team" attacker to find vulnerabilities in an image classification model. Second, act as a "Blue Team" defender to secure the model against attacks. Finally, package the entire project safely using **Docker**.

2. Red Team Phase: Finding Model Vulnerabilities

Before securing our AI, we needed to test how fragile it was. We focused on **Adversarial Evasion Attacks**, where an attacker tries to trick the AI into making a wrong prediction by making tiny, hidden changes to an input image.

A. The FGSM Attack (Fast Gradient Sign Method)

- **What we did:** We used FGSM to add calculated noise to pictures of handwritten digits.



Original Image



Adversarial

- **The Setting ($\epsilon = 0.3$):** Epsilon (ϵ) is our noise budget. Setting it to 0.3 means we allowed pixels to change up to 30% of their total intensity range.
- **The Result:**

```
(.venv)
chala@DESKTOP-MH5LDT MINGW64 ~/Desktop/AIRedTeam (main)
$ python attack.py
[*] True Label: 0
-> Original Model Prediction: 0
-> Adversarial Model Prediction (FGSM): 3 (Evasion Success: True)
```

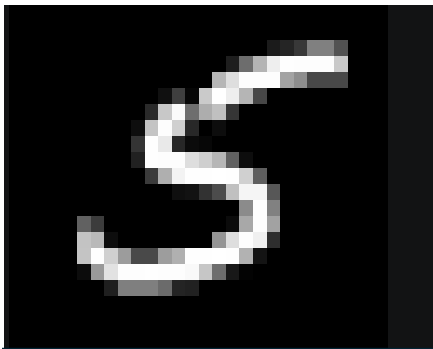
AI prediction

To a human eye, the image still looked like the correct number. However, the AI model was completely fooled and misclassified the image. This proved the model had a high-risk evasion vulnerability.

B. Testing Stronger Attacks (PGD)

To test the absolute limits of our AI, we also looked at **Projected Gradient Descent (PGD)** parameters:

- $\text{EPS} = 0.3$ (Total noise limit)
- $\text{ALPHA} = 2 / 255$ (The size of each tiny step the attack takes)
- $\text{STEPS} = 10$ (How many times the attack repeats)
- PGD is a multi-step version of FGSM and is considered one of the strongest tests for AI model robustness.



Original



Adversarial

```
(.venv)
chala@DESKTOP-MH5LDT MINGW64 ~/Desktop/AIRedT
$ python pgd_attack.py
[*] Loading model: model.pth
[*] Searching for correctly classified image..
[*] True Label: 5
[*] Original Model Prediction: 5
[*] Generating PGD adversarial example...
-> Original Prediction: 5
-> PGD Prediction: 2
-> Evasion Success: True
[*] Saved: pgd_original_sample.png
[*] Saved: pgd_adversarial_sample.png
[*] PGD attack completed successfully!
```

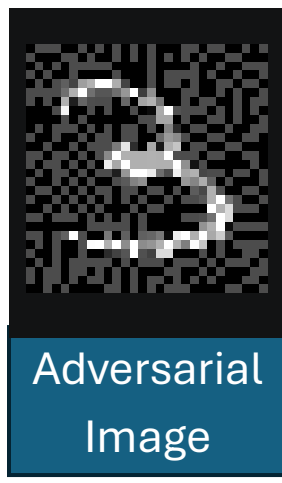
Prediction

3. Defending and Hardening the Model

To fix the vulnerabilities found by the Red Team, we implemented a defense mechanism called **Adversarial Training**.

- **How Adversarial Training Works:** Instead of training our neural network (SimpleCNN) only on normal, clean images, we updated our training function (train_robust_model). During training, the script generates FGSM adversarial images on-the-fly and forces the model to learn from them.
- **Why it helps:** By exposing the AI to these tricky inputs while it is learning, it stabilizes its internal decision boundaries. This makes it much harder for attackers to fool the model with simple pixel noise later on.

i. **Defended FGSM attack:**



```
(.venv)
chala@DESKTOP-MH5LDT MINGW64 ~/Desktop/AIRedT
$ python attack.py
[*] Loading model for adversarial attack...
[*] True Label: 3
-> Original Model Prediction: 3
-> Adversarial Model Prediction (FGSM): 3 (Eva
[*] Saved image to: original_sample.png
[*] Saved image to: adversarial_sample.png
[*] Success!
```

Predicted
result

ii. Defended PGD attack:



```
(.venv)
chala@DESKTOP-MHMSLDT MINGW64 ~/Desktop/AIRedT
$ python pgd_attack.py
[*] Original Model Prediction: 7
[*] Generating PGD adversarial example...
-> Original Prediction: 7
-> PGD Prediction: 7
-> Evasion Success: False
[*] Saved: pgd_original_sample.png
[*] Saved: pgd_adversarial_sample.png
[*] PGD attack completed successfully!
```

4. Infrastructure Phase: Safe Deployment with Docker

To ensure our code, model weights, and python libraries run reliably on any computer without environment issues, we containerized the project using Docker.

- **The Docker Setup (Dockerfile):**

- We used a lightweight and secure base image: **python:3.9-slim**.
- We set a working directory (/app), copied our requirements.txt, and installed all required machine learning libraries.
- We copied all project files (including our trained model weights model.pth) directly into the container.

- **Why Docker Matters:** It ensures that anyone running our project gets the exact same secure, isolated environment, eliminating standard setup errors.

5. Conclusion

This project successfully demonstrated the full security lifecycle of an AI application:

1. **Discovered** that standard deep learning models are easily tricked by subtle adversarial noise (FGSM/PGD).
2. **Mitigated** the risk by implementing an adversarial training defense loop in PyTorch.
3. **Secured** the deployment workflow by packaging everything into a clean, portable Docker container.